CM3035 Coursework 1: Chicago Crime REST API

Module: CM3035 Advanced Web Development **Author:** Matthias Naumann **Date:** June 2026

---

1. Introduction and Dataset

This report describes a Django REST API serving Chicago crime incident data. The app loads a CSV into a relational database, exposes six analytical endpoints, and ships with a 33-test suite that includes property-based testing.

The dataset comes from the City of Chicago Open Data Portal's "Crimes (2001 to Present)" collection, filtered to the 9,500 most recent records. Chicago's data works well here for one reason: it packs three analytical dimensions into a single dataset. Each incident has a precise timestamp (temporal), a typed offense classification (categorical), and a police district assignment (geographic). That structure lets queries reach past basic CRUD.

The 9,499 incidents split across 27 offense categories and 22 police districts. Theft tops the list at 2,024 incidents, then battery (1,902), criminal damage (1,067), assault (848), and motor vehicle theft (791). Each row carries a case number, date, address block, arrest flag, domestic flag, FBI classification code, and foreign keys into both offense category and district.

---

2. Database Design

Three Django models: Incident, OffenseCategory, District. Same shape as the gene-to-EC-to-sequencing relationship from Topic 4. The split works because offense categories and districts are independent entities, while each incident references both.

OffenseCategory

Stores offense classification data. The `code` field holds the IUCR (Illinois Uniform Crime Reporting) code, unique per offense type. The `category` field slots each offense into one of three groups: violent, property, or quality-of-life. That classification is what makes the aggregation queries in Section 3 useful.

District

A police district. Unique district number, name, area type (police district or community area), and a population figure. The population field is what lets the district safety endpoint compute per-capita rates. Without it, a small district with 10 incidents would automatically rank above a large district with 50.

Incident

The central fact table. Each incident links to one offense category and one district via foreign keys with `on_delete=CASCADE` and `related_name='incidents'`. Reverse lookups (`OffenseCategory.incidents.count()`, `District.incidents.filter(arrest=True)`) keep the aggregations cheap. Cascade deletion cleans up neatly when a category or district goes.

The shape mirrors the course's two-FK pattern, applied to a domain with more analytical dimensions.

---

3. API Endpoints

Six REST endpoints, all linked from the HTML root at `/api/`. Each answers a specific question, and the queries grow more complex going down the list.

Endpoint 1: Incident List: `GET /api/incidents/`

Paginated list with nested offense category and district data. Filtering by category name, district name, date range, and arrest flag via query parameters. Page size is 100 by default, and the default ordering is date descending so the freshest incidents surface first. Filters combine with AND: `?category=theft&arrest=true` returns theft incidents where an arrest was made. The view uses `select_related('primary_type', 'district')` so serialization avoids the N+1 query trap.

```
queryset = Incident.objects.select_related('primary_type', 'district').all()
if category:
    queryset = queryset.filter(primary_type__name__icontains=category)
if date_from:
    queryset = queryset.filter(date__gte=date_from)
if arrest is not None:
    queryset = queryset.filter(arrest=(arrest.lower() == 'true'))
```

Endpoint 2: Incident Detail: `GET /api/incidents/{id}/`

Single incident, fully nested with category and district. Supports PUT and DELETE, returns 200/201/204 on success and 404 on a missing id.

Endpoint 3: Create Incident: `POST /api/incidents/`

Takes JSON with `primary_type_id` and `district_id` as write-only integers. The serializer's `create()` validates those ids exist, raises `ValidationError` with field-specific messages if not, then links the new incident. Matches the Topic 4 POST pattern.

Endpoint 4: Arrest Rates: `GET /api/stats/arrest-rates/`

Arrest rate percentage by offense category, sorted so the lowest-resolution categories float to the top. The query uses Django's conditional aggregation: `Count` with a `Q` filter for arrest status, wrapped in a `Case`/`When` to keep division safe when a category has zero incidents.

```
OffenseCategory.objects.annotate(
    total=Count('incidents'),
    arrests=Count('incidents', filter=Q(incidents__arrest=True)),
).annotate(
    arrest_rate=Case(
        When(total=0, then=0.0),
        default=100.0 * F('arrests') / F('total'),
    )
).values('name', 'category', 'total', 'arrests', 'arrest_rate')
    .order_by('-arrest_rate')
```

The interesting thing is what the query reveals about resolution rates per offense type. Raw counts don't. Sorting the lowest arrest rates to the top surfaces the offense types police are least able to resolve. On the Chicago data, criminal trespass and deceptive practice float high on the non-resolution list, which matches what those categories look like in practice: hard to catch in the act.

Endpoint 5: District Safety: `GET /api/stats/district-safety/`

Per-capita incident rate for each police district (incidents per 100,000 residents), highest first. The 100,000 multiplier turns per-capita rates into a more readable figure. District populations sit between 30,000 and 56,000, so without the multiplier the raw per-capita values would read as tiny decimals. Uses `F()` expressions for field arithmetic plus a `Case` guard for zero-population districts. The guard matters because a freshly-created district with no population set would otherwise divide by zero and 500 the request. Surface question: which districts have the most crime relative to their population?

Endpoint 6: Temporal Trends: `GET /api/stats/temporal/`

Aggregates by month with `TruncMonth`, returning monthly totals and arrest percentages in chronological order. Monthly granularity sits in the sweet spot between showing every day's noise and showing only yearly totals. At finer resolution, crime patterns look like spikes. Coarser resolution hides the seasonal cycles. Picks up seasonal patterns and tracks whether arrest rates shift over time.

Main Page

`/api/` renders an HTML page with all six endpoints as clickable links, plus system metadata: Python version, Django version, installed packages, and admin credentials. Satisfies the assignment's hyperlinked endpoint and metadata requirements.

---

4. Implementation

Project Structure

Code is organised the same way as Topics 3 and 4 of the course. REST views live in `api.py`, separate from any HTML views. Serializers in `serializers.py`, fixtures in `model_factories.py`, tests split across a `tests/` package: `test_api.py` for endpoint integration, `test_serializers.py` for isolated serializer tests plus the Hypothesis property-based test. One file, one responsibility.

Django REST Framework Patterns

All six endpoints use class-based views. `IncidentList` extends `ListCreateAPIView`, combining list and create in a single class with a custom `get_queryset()` for filtering. `IncidentDetail` extends `RetrieveUpdateDestroyAPIView` for full CRUD on a single row. The three statistical endpoints extend `GenericAPIView` with custom `get()` methods, since their response shapes don't map to a queryset-and-serializer pattern.

Serializers follow ModelSerializer. `IncidentSerializer` uses nested read-only serializers for `primary_type` and `district` (GETs include the related objects fully), and write-only integer fields for `primary_type_id` and `district_id` (POSTs just pass id references). The overridden `create()` resolves those ids and raises descriptive validation errors on bad input. Same pattern as the Topic 4 POST lesson.

CSV Data Loading

Loading runs as a Django management command (`python manage.py load_crime_data <csv_file>`). It reads the CSV with `csv.DictReader`, creates offense categories and districts via `get_or_create`, classifies each offense type by keyword into violent / property / quality-of-life, and writes incidents with timezone-aware datetimes. Capped at 9,500 rows to stay under the 10,000-entry assignment limit.

The command is idempotent on `(case_number, district_num)`. `get_or_create` on categories and districts plus the unique constraint on incident case numbers means re-running with the same CSV after a partial failure picks up where it left off, no manual cleanup needed. The loader also forces the C locale before parsing AM/PM timestamps. Linux servers default to a locale that won't recognise the `%p` format specifier and silently fail the parse, which was the original bug here. Force-set to C and the timestamps parse consistently across Windows, macOS, and Linux.

Swagger Documentation

`drf-spectacular` generates an OpenAPI 3.0 schema at `/api/schema/` and a Swagger UI at `/api/docs/`. All six endpoints are documented with their HTTP methods, parameters, and response schemas. Goes beyond the course content.

---

5. Testing Strategy

33 tests across two files, drawing on the Topic 4 testing lessons.

Test Fixtures

`model_factories.py` defines FactoryBoy factories for all three models using `Sequence` for unique fields, `Faker` for realistic data, `choice` for constrained fields, and `randint` for numeric ranges. Every run uses randomised data rather than static fixtures, so the suite handles varied input without special-casing.

API Integration Tests (`test_api.py`)

Six `APITestCase` classes, one per endpoint: successful responses (200, 201, 204), error responses (400 for missing fields, 400 for invalid foreign keys, 404 for missing resources), filtering behaviour (category, arrest status), response structure (nested serialised objects present), and calculation correctness (arrest rates, per-capita values, temporal aggregation). Each class uses `setUp` to seed controlled data and `tearDown` to clean up.

Serializer Tests (`test_serializers.py`)

Isolated serializer tests verify that `IncidentSerializer` accepts valid data, rejects missing required fields, correctly links foreign keys in `create()`, and raises `ValidationError` for nonexistent FKs. Separate tests cover `OffenseCategorySerializer` and `DistrictSerializer` field presence and validation.

Hypothesis Property-Based Testing

The Hypothesis test exercises `IncidentSerializer` with 50 randomly generated valid input combinations (varying case numbers, dates, blocks, arrest and domestic flags, FBI codes). Each generated input either validates and saves, or fails with structured errors. Either way the serializer doesn't crash. Hypothesis pairs cleanly with FactoryBoy here: FactoryBoy generates the parent records (OffenseCategory, District) once per test, then Hypothesis generates leaf values for each incident. The leaf-level randomness catches edge cases the manual tests don't: empty strings, dates at the calendar boundaries, special characters in block names, unicode text. Beyond the course syllabus, fits the "advanced techniques" criteria.

```
@given(
    case_number=st.text(min_size=1, max_size=18),
    date_str=st.dates().map(lambda d: d.isoformat() + 'T12:00:00Z'),
    block=st.text(min_size=1, max_size=190),
    arrest=st.booleans(),
    domestic=st.booleans(),
    fbi_code=st.text(min_size=1, max_size=8),
)
@settings(max_examples=50, deadline=None)
def test_serializer_never_crashes_on_varied_input(self, ...):
    ...
```

---

6. Critical Evaluation

This was built against a coursework spec with tight constraints: SQLite3, single Django server, no auth. A production-grade crime API would need more.

Database engine. SQLite3 fits development and small datasets, but a production service with concurrent writes wants PostgreSQL for row-level locking, connection pooling, and geographic query support (PostGIS for spatial filtering by district boundary).

Authentication and authorisation. The API uses DRF's `AllowAny` permission class, so every endpoint is public. Production would want token or session auth, rate limiting, and differentiated access: public for aggregate stats, authenticated for incident-level data.

Asynchronous processing. CSV loading runs synchronously inside a management command. At scale this would block the server. Production should move ingestion to an async task queue (Celery, per Topic 6) with progress reporting and error recovery.

Pagination and performance. The incident list uses page-number pagination at 100 records per page. Fine for 9,500 records (95 pages total). At 100k records that becomes 1,000 pages, and the offset-based query slows on later pages because the database has to walk through every preceding row. Cursor pagination (keyset on date + id) would fix that. Indexes on the four filtered columns would also help: the current schema declares no explicit indexes, so each filter does a full scan until the database builds query statistics.

Continuous integration. Tests run locally via `python manage.py test`. A CI pipeline (GitHub Actions, for instance) would run the suite on every push, enforce style with flake8 or ruff, and block deployment on failure so only verified code reaches users.

District population values. Chicago's police districts do not publish per-district population figures, so each `District` row's `population` field is seeded with a US Census ACS community-area estimate used as a proxy (City of Chicago Open Data Portal, "ACS 5-Year Data by Community Area"). This is a known limitation of the `District Safety` endpoint: absolute per-capita values are illustrative rather than authoritative, and ranking across districts reflects the relative ordering of community-area populations rather than the actual resident counts served by each police district. Acknowledging the source rather than fabricating numbers is the defensible choice for a coursework submission.

None of these gaps block a coursework submission. The app demonstrates the core concepts (RESTful design, ORM queries, serialisation, testing), and this section calls out what production would change.

---

7. Run Instructions

Environment

- Operating System: Windows 11 Enterprise (also tested on Linux/macOS)
- Python version: 3.12.1
- Django version: 5.0.7
- Key packages: djangorestframework 3.15.2, drf-spectacular 0.28.0, factory-boy 3.3.1, hypothesis 6.111.1

Setup

```
# 1. Extract the ZIP archive
unzip cm3035_coursework_submission.zip
cd coursework2026

# 2. Create and activate a virtual environment
python -m venv venv
venv\Scripts\activate      # Windows
# source venv/bin/activate  # Linux/macOS

# 3. Install dependencies
pip install -r requirements.txt

# 4. Apply database migrations
python manage.py migrate

# 5. Load the dataset
```

```
python manage.py load_crime_data chicago_crime_data.csv

# 6. Run the tests
python manage.py test crime_api -v 2

# 7. Start the development server
python manage.py runserver
```

Access

- API root: http://localhost:8000/api/
- Swagger docs: http://localhost:8000/api/docs/
- Admin site: http://localhost:8000/admin/
- Admin credentials: username `admin`, password `admin123`

References

City of Chicago Open Data Portal (2026). *Crimes (2001 to Present).* Available at: https://data.cityofchicago.org/Public-Safety/Crimes-2001-to-Present/ijzp-q8t2

Fielding, R.T. (2000). *Architectural Styles and the Design of Network-based Software Architectures.* PhD Thesis, University of California, Irvine.

Django REST Framework Documentation. Available at: https://www.django-rest-framework.org/

Hypothesis Documentation. Available at: https://hypothesis.readthedocs.io/